EX.NO : 01 08/07/2025	ARRAY BASED LIST AOT USING C

PROBLEM STATEMENT :

A small library stores the list of newly arrived books in an array-based structure. The librarian wants to maintain the order of arrival. Given the arrival sequence: “AI Basics”, “Data Science”, “Python 101”, “Algorithms”, “Machine Learning”, Insert them into an array-based List ADT. A book “Cyber Security” arrives and must be inserted at position 3. Later, “Python 101” is removed as it's damaged. Design the array-based List ADT operations to update the list and display the final order.

AIM :

To perform insert and delete operations on an army-based. List ADT using c

ALGORITHM :

1. start

2. Create a list of boots:

- Al Basics
- Dala scieve
- Python 101
- Algorithms
- Machine Learning

3. Insert "Cyber Security" at position 3.

4. Delete "Python 101 "from the list.

5. Display the updated list.

6. Stop.

SOURCE CODE :

```
#include <stdio.h>

#include <string.h>

int main()

{

   char books [6][30]={

            "Al Basics",

            “Data scorce",
```

```
            "Python 101",
            "Algorithms"
           "Machine Learoung"
};
int i;
for (i=5; 1>2; i--)
      strcpy (books[i], books [i-1]);
      strcopy (books [2], "Cyber sewnity");
for (i=3;i<5; i++)
     strepy (books [1], books [i+D);
     printf("Updated Book List:\n");
for (i=0;i<5; i++)
     printf("%s\n', books[i]);
     return 0;
}
```

OUTPUT :

Modified Book List

1. AI Basies
2. Data Science
3. Cyber Security
4. Algorithms
5. Machine Learning

Preparation (5)	
Implementation (5)	
Viva Voce (5)	
Output & Result (5)	
TOTAL (20)	

RESULT :

Thus, the insert operation (adding Cyber Security of position 3) and the delete operation (removing Python 101) were successfully performed using an allry-based List ADT in C.

EX.NO : 02 15/07/2025	**Implementation of Lindred list AOT To maintain a dynamic hospital waiting list ausing python.**

PROBLEM STATEMENT :

A hospital needs to maintain a waiting list of patients dynamically as patients arrive and get treated. Insert patients: John, Priya, Abdul, Meera in order. A senior citizen “Ravi” should be added after Priya. “Abdul” leaves before consultation. Using Linked List ADT, show how the list changes after each operation.

Aim :

To implement a Linked List ADT in Python for maintaining a hospital writing list by performing insertion, invertion after a specific patient, and deletion operating and to display the list after each operation.

ALGORITHM :

1. Create a Node dass with data and next pointer
2. Create a LinkedList dass.
3. Insert patients John, Priya, Abdul, and Meera at the end of the list.
4. Display the waiting list.
5. Insert Ravi after Priya
6. Display the updated waiting list.
7. Delete Abdul from the list
8. Display the final waiting list.
9. End the program.

SOURCE CODE :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
```

```
def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    def insert_after(self, key, data):
        temp = self.head
        while temp:
            if temp.data == key:
                new_node = Node(data)
                new_node.next = temp.next
                temp.next = new_node
                return
            temp = temp.next
    def delete(self, key):
        temp = self.head
        if temp and temp.data == key:
            self.head = temp.next
            return
        prev = None
        while temp and temp.data != key:
            prev = temp
            temp = temp.next
        if temp is None:
            return
```

```
        prev.next = temp.next
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end="->")
            temp = temp.next
        print("None")

# Main Program remains the same
patients = LinkedList()
print("After inserting John:")
patients.insert_end("John")
patients.display()
print("\nAfter inserting Priya:")
patients.insert_end("Priya")
patients.display()
print("\nAfter inserting Abdul:")
patients.insert_end("Abdul")
patients.display()
print("\nAfter inserting meera:")
patients.insert_end("meera")
patients.display()
print("\nAfter inserting Ravi and Priya:")
patients.insert_after("Priya", "Ravi")
patients.display()
print("\nAfter deleting Abdul:")
patients.delete("Abdul")
patients.display()
```

OUTPUT :

After inserting John:

John->None

After inserting Priya:

John->Priya->None

After inserting Abdul:

John->Priya->Abdul->None

After inserting meera:

John->Priya->Abdul->meera->None

After inserting Ravi and Priya:

John->Priya->Ravi->Abdul->meera->None

After deleting Abdul:

John->Priya->Ravi->meera->None

Preparation (5)	
Implementation (5)	
Viva Voce (5)	
Output & Result (5)	
TOTAL (20)	

RESULT :

Thus, the Linked List ADT was succesfully implemented in Python to manage the hospital waiting list. The insertion, insertion after a specific patient, and deletion operations were performed successfully, and the list was displayed correctly ofter each operations.

EX.NO : 03 22/07/2025	**Implementation of airport check in System using Circular Queve using python.**

PROBLEM STATEMENT :

An airport check-in counter processes passengers using a circular queue system. Passengers arriving in order: P1, P2, P3, P4, P5. The queue size is 4 due to staff shortage. · Enqueue all passengers in order and show how overflow occurs. Two passengers are processed (dequeued). P6 and P7 arrive. Simulate these operations using array-based circular queue and show rear/front updates.

Aim:

To implement an array-based circular queve in pythin and simulate airport check-in operations with queve size 4, showing enqueve, Lequeve, overflow, and front/rear uplates.

Algorithm:

1. Initialize the circular queve with size 4.

2. Set Front =-1 and rear=-1.

3. Enqueve passengers P1, P2, P3, P4.

4. Attempt to enqueve P5.

   - if (rear +1) % Size == front, display Queve overflow.

5. Dequeve tavo passengers.

6. Enqueve Pb and P7.

7. After every operation, display

   - Queve contents
   - Front indesc
   - Rear index

8. stop

Code:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
```

```
def enqueue(self, item):
    if (self.rear + 1) % self.size == self.front:
        print(f"{item} -> Queue Overflow")
        return
    if self.front == -1:
        self.front = 0
        self.rear = 0
    else:
        self.rear = (self.rear + 1) % self.size
    self.queue[self.rear] = item
    print(f"Enqueued: {item}")
    self.display()
def dequeue(self):
    if self.front == -1:
        print("Queue Underflow")
        return
    item = self.queue[self.front]
    self.queue[self.front] = None
    if self.front == self.rear:
        self.front = -1
        self.rear = -1
    else:
        self.front = (self.front + 1) % self.size
    print(f"Dequeued: {item}")
    self.display()
def display(self):
    print("Queue:", self.queue)
    print("Front =", self.front, "Rear =", self.rear)
    print("-" * 40)
```

```
cq = CircularQueue(4)
print("Enqueue Passengers:")
cq.enqueue("P1")
cq.enqueue("P2")
cq.enqueue("P3")
cq.enqueue("P4")
cq.enqueue("P5")
print("\nProcess Two Passengers:")
cq.dequeue()
cq.dequeue()
print("\nNew Arrivals:")
cq.enqueue("P6")
cq.enqueue("P7")
```

Output:

```
Enqueue Passengers:
Enqueued: P1
Queue: ['P1', None, None, None]
Front = 0 Rear = 0
Enqueued: P2
Queue: ['P1', 'P2', None, None]
Front = 0 Rear = 1
Enqueued: P3
Queue: ['P1', 'P2', 'P3', None]
Front = 0 Rear = 2
Enqueued: P4
Queue: ['P1', 'P2', 'P3', 'P4']
Front = 0 Rear = 3
P5 -> Queue Overflow
```

Process Two Passengers:

Dequeued: P1

Queue: [None, 'P2', 'P3', 'P4']

Front = 1 Rear = 3

Dequeued: P2

Queue: [None, None, 'P3', 'P4']

Front = 2 Rear = 3

New Arrivals:

Enqueued: P6

Queue: ['P6', None, 'P3', 'P4']

Front = 2 Rear = 0

Enqueued: P7

Queue: ['P6', 'P7', 'P3', 'P4']

Front = 2 Rear = 1

Preparation (5)	
Implementation (5)	
Viva Voce (5)	
Output & Result (5)	
TOTAL (20)	

Result:

The may-based Circular Queve was Successfully implemented in Python, overflow occurred when attempting to insert P5 into the full queve. After processing two passengers, the queve reused the treed positions by wrapping wround, allowing Pt and P7 to be inserted Successfully while correctly updating the pront and Year indicos.

EX.NO : 04 29/07/2025	Implementation of Queue Using Linked List in Python for Bank Customer Service

PROBLEM STATEMENT :

A bank uses a linked-list–based queue for serving customers. Customers: A, B, C arrive. Then D arrives. B and C are served. Implement the queue operations and show the dynamic linked list changes.

Aim:

a Linked List in To implement a queve using Python and perform congreve and dequeve operations.

Algorithm :

1. Create an empty queve.
2. Enqueve A, B, C, and D.
3. Dequeve two customers.
4. Display the queve.

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, data):
        new = Node(data)
        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new
        print(data, "Inserted")
```

```
    def dequeue(self):
        if self.front is None:
            print("Queue is Empty")
            return
        print(self.front.data, "Served")
        self.front = self.front.next
        if self.front is None:
            self.rear = None
    def display(self):
        temp = self.front
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

q = Queue()
q.enqueue("A")
q.enqueue("B")
q.enqueue("C")
q.enqueue("D")
print("\nQueue:")
q.display()
print("\nDequeuing Two Elements:")
q.dequeue()
q.dequeue()
print("\nQueue After Dequeue:")
q.display()
```

Output:

A Inserted

B Inserted

C Inserted

D Inserted

Queue:

A -> B -> C -> D -> None

Dequeuing Two Elements:

A Served

B Served

Queue After Dequeue:

C -> D -> None

Preparation (5)	
Implementation (5)	
Viva Voce (5)	
Output & Result (5)	
TOTAL (20)	

Result:

The queve was successfully implemented using a linked using list in Python. The customer were served in FIFO order leaving the queve as: C→D.

EX.NO : 05 05/08/2025	Bubble Sort and Insertion Sort using Python

PROBLEM STATEMENT :

A sports event records running times of participants. The unsorted times are 15.2, 12.8, 20.5, 11.6, 19.3. The coach wants to sort the times to shortlist top performers. Apply Bubble Sort and Insertion Sort separately to sort the timings. Show the status of the list after each pass for both algorithms.

Aim

To sort the running times of participants in ascending order using Bubble Sort and Insertion Sort, and display the list after each pass.

Algorithm

Bubble Sort Algorithm

1. Read the list of running times.
2. Compare each pair of adjacent elements.
3. If the left element is greater than the right element, swap them.
4. Repeat the process for all elements.
5. After each pass, the largest element moves to its correct position.
6. Continue until the list is completely sorted.

Insertion Sort Algorithm

1. Read the list of running times.
2. Assume the first element is already sorted.
3. Take the next element as the key.
4. Compare the key with previous elements.
5. Shift larger elements one position to the right.
6. Insert the key into its correct position.
7. Repeat until all elements are sorted.

Code:

```
# Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    print("Bubble Sort:")
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
        print(f"Pass {i + 1}: {arr}")
# Insertion Sort
def insertion_sort(arr):
    print("\nInsertion Sort:")
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
        print(f"Pass {i}: {arr}")

# Main Program
times = [15.2, 12.8, 20.5, 11.6, 19.3]
bubble_list = times.copy()
insertion_list = times.copy()
bubble_sort(bubble_list)
insertion_sort(insertion_list)
```

Output

Bubble Sort:

Pass 1: [12.8, 15.2, 11.6, 19.3, 20.5]

Pass 2: [12.8, 11.6, 15.2, 19.3, 20.5]

Pass 3: [11.6, 12.8, 15.2, 19.3, 20.5]

Pass 4: [11.6, 12.8, 15.2, 19.3, 20.5]

Insertion Sort:

Pass 1: [12.8, 15.2, 20.5, 11.6, 19.3]

Pass 2: [12.8, 15.2, 20.5, 11.6, 19.3]

Pass 3: [11.6, 12.8, 15.2, 20.5, 19.3]

Pass 4: [11.6, 12.8, 15.2, 19.3, 20.5]

Preparation (5)	
Implementation (5)	
Viva Voce (5)	
Output & Result (5)	
TOTAL (20)	

Result

The running times were successfully sorted in **ascending order** using both **Bubble Sort** and **Insertion Sort**.